Personal Finance App

MVP Product Requirements Document (PRD)

Document Version: 1.0
Date: November 10, 2026
Status: Product Scope Defined — Ready for Architecture Planning

---

1. Product Overview

1.1 Product Vision

Build a secure, multi-user Personal Finance Management application that helps individuals manage their finances from one place.

The application will provide users with tools to track income, expenses, accounts, transfers, budgets, savings goals, credit cards, debts, loans, bills, subscriptions, and financial performance.

The product will initially be completely free and designed as a Progressive Web App (PWA), supporting desktop and mobile experiences, including offline functionality and automatic synchronization.

The long-term vision is to evolve the application into an intelligent financial assistant powered by AI.

1.2 Product Goals

The MVP aims to:

- Give users a complete overview of their financial position.
- Track financial transactions accurately.
- Support multiple accounts and currencies.
- Calculate total assets, liabilities, and net worth.
- Manage budgets, savings goals, debts, and credit cards.
- Support recurring financial activities.
- Provide useful monthly financial reports.
- Work reliably on mobile and desktop.
- Support offline financial activity with automatic synchronization.
- Protect users' financial information through strong security and data isolation.

1.3 Target Users

Individuals who want to:

- Track their personal income and expenses.
- Manage multiple financial accounts.
- Monitor credit cards and outstanding debts.
- Organize monthly budgets.
- Save toward financial goals.
- Understand their financial position.
- Manage bills and recurring payments.

---

2. Product Scope

2.1 MVP Features

The MVP will include:

1. User Authentication and Authorization.
2. Financial Account Management.
3. Multi-Currency Support.
4. Transaction Management.
5. Categories Management.
6. Money Transfers.
7. Savings Goals.
8. Credit Card Management.
9. Debt and Loan Management.
10. Budget Management.
11. Recurring Transactions.
12. Bills and Subscriptions.
13. Financial Dashboard.
14. Monthly Financial Reports.
15. Notifications.
16. Global Search.
17. Offline Support and Automatic Synchronization.
18. Data Export.
19. PWA Installation Support.
20. Light and Dark Themes.
21. Administrative Capabilities.

2.2 Deferred Features

The following features are outside the MVP:

- AI CFO Assistant.
- Financial recommendations powered by AI.
- Cash flow predictions.
- Automated financial decisions.
- Family and household financial sharing.
- User-facing account deletion.
- Manual exchange-rate overrides.
- Advanced debt amortization calculations.
- Annual and custom-range financial reports.
- Paid subscriptions and premium plans.
- Two-factor authentication.
- User-facing backup and restore.
- Customizable dashboard layouts.

---

3. User Management and Authorization

3.1 Multi-User Architecture

The application will support multiple independent users from day one.

Each user manages their own financial information.

All financial records must be associated with the appropriate user.

Users must never be able to access, modify, or delete another user's financial records.

3.2 Roles

User

Can:

- Manage personal accounts.
- Create and manage transactions.
- Manage budgets.
- Manage savings goals.
- Manage credit cards.
- Manage debts and loans.
- Manage bills and subscriptions.
- View financial reports.

- Export personal data.
- Configure notification preferences.

Admin

Can:

- Manage users.
- Deactivate user accounts.
- Manage system settings.
- Monitor application errors.
- Monitor background jobs.
- Monitor system health.

Security requirement: Admin privileges must not automatically provide access to users' private financial details.

3.3 Account Deactivation

The MVP will support account deactivation.

A user-facing account deletion feature will not be provided in the MVP.

However, account deactivation must not be treated as equivalent to permanent deletion. Data retention, privacy obligations, and applicable legal requirements must be addressed separately.

---

4. Authentication and Security

4.1 Authentication Methods

The application will support:

- Email and Password.
- Google Login.
- Email Verification.
- Forgot Password.
- Secure Sessions.
- Logout.

Two-factor authentication is deferred.

4.2 Security Requirements

The system must implement:

- Secure password hashing.
- OAuth authentication.
- Secure session management.
- Authorization checks.
- Rate limiting.
- Email verification.
- Secure password reset.
- HTTPS communication.
- Protection against unauthorized data access.
- Secure handling of secrets and API credentials.

Financial data must be protected both in transit and at rest.

---

5. Financial Account Management

5.1 Account Types

Users can create:

- Cash Accounts.
- Bank Accounts.
- Credit Cards.
- Savings Accounts.
- Digital Wallets.

Examples of Digital Wallets include Vodafone Cash.

5.2 Account Creation

Each account must support:

- Account Name.
- Account Type.
- Currency.
- Opening Balance.
- Creation Date.
- Account Status.

5.3 Opening Balance

Opening Balance represents the amount of money available when a user starts using the application.

It is not considered Income.

The application must maintain a clear distinction between opening balances and actual financial transactions.

5.4 Balance Management

The application will use stored current balances.

Balances must be updated whenever a financial transaction is created, edited, or deleted.

All transaction and balance modifications must happen atomically inside database transactions.

The system must maintain transaction history and audit information.

Account balances are displayed in their original account currencies.

Aggregated financial information is displayed in the user's Base Currency.

---

6. Currency Management

6.1 Supported Currencies

Initial supported currencies:

- EGP.
- USD.
- EUR.

6.2 Base Currency

Each user selects one Base Currency.

The Base Currency is used for:

- Total Assets.
- Total Liabilities.
- Net Worth.
- Financial Reports.

- Budgets.
- Financial Dashboard.

6.3 Exchange Rates

The application will use an automatic exchange-rate API.

Exchange rates will be refreshed once daily.

The application will use market exchange rates.

Manual exchange-rate overrides are not included in the MVP.

6.4 Historical Exchange Rates

For backdated transactions, the application must use the historical exchange rate corresponding to the transaction date.

The exchange rate used must be stored with the transaction.

The application must preserve:

- Original Amount.
- Original Currency.
- Exchange Rate Used.
- Converted Amount.
- Conversion Date.

Historical reports must not change simply because exchange rates are updated later.

6.5 Exchange Rate API Failure

If the exchange-rate provider becomes unavailable:

- Use the last successfully cached exchange rate.
- Clearly indicate that the rate is stale.
- Display the last successful update time.
- Do not silently present stale data as current.
- Do not block users from entering transactions.

---

7. Transaction Management

7.1 Transaction Types

The application will support four transaction types:

Income

Money received by the user.

Examples:

- Salary.
- Freelance Income.
- Business Income.
- Refunds.

Expense

Money spent by the user.

Examples:

- Food.
- Transportation.
- Healthcare.
- Housing.
- Entertainment.

Transfer

Money moved between accounts owned by the same user.

Transfers must not be counted as Income or Expenses.

Adjustment

A balance correction used when the actual account balance differs from the recorded balance.

Adjustments must maintain an audit trail.

7.2 Transaction Fields

Each transaction must support:

- Amount.
- Currency.
- Date and Time.
- Account.
- Category.
- Description or Notes.
- Transaction Type.

Additional technical fields will support synchronization, auditing, and financial calculations.

7.3 Transaction Operations

Users can:

- Create transactions.
- View transactions.
- Edit transactions.
- Soft-delete transactions.

New transactions affect account balances immediately.

7.4 Transaction Editing

When a transaction is edited, the system must correctly reverse its previous financial impact and apply the updated impact.

All operations must be atomic.

7.5 Transaction Deletion

Transactions will use Soft Delete.

The application must preserve audit information.

The audit trail must record:

- Previous Values.
- New Values.
- Actor.
- Timestamp.
- Operation Type.

7.6 Refunds

Refunds will be recorded as Income using a predefined category named `Refunds`.

In the MVP:

- Refunds appear in Income.
- Refunds do not modify the original Expense.
- Refunds do not reduce Budget consumption.

7.7 Transaction Attachments

Supported attachments:

- JPG.
- PNG.
- WebP.
- PDF.

Maximum file size: 10 MB per file.

Attachments must be stored in private Object/File Storage rather than directly inside the database.

Users can:

- Upload attachments.
- View attachments.
- Download attachments.
- Delete attachments.

The system must validate file type, file size, and access permissions.

---

8. Categories Management

8.1 Predefined Categories

Initial examples:

- Food.
- Transportation.
- Housing.
- Healthcare.
- Salary.
- Freelance.

- Entertainment.
- Refunds.

8.2 Custom Categories

Users can create their own categories.

Custom categories must be private to the user who created them.

Categories must support appropriate transaction classification.

---

9. Transfers

Users can transfer money between their own accounts.

Transfers must:

- Decrease the source account balance.
- Increase the destination account balance.
- Support different currencies.
- Use the applicable exchange rate.
- Preserve the original transfer details.
- Avoid counting transfers as Income or Expenses.

Credit card payments will also be represented as Transfers.

This prevents the same financial activity from being counted twice.

---

10. Savings Goals

10.1 Savings Goal Fields

Each goal must include:

- Goal Name.
- Target Amount.
- Currency.
- Current Saved Amount.
- Optional Target Date.
- Progress Percentage.
- Status.

10.2 Goal Status

Supported statuses:

- Active.
- Completed.
- Paused.

10.3 Funding Savings Goals

Users fund savings goals through Transfers.

The user selects:

- Savings Goal.
- Source Account.
- Transfer Amount.

The system updates the source account balance, savings account balance, and goal progress.

Savings transfers are not Expenses and do not reduce total assets.

---

11. Credit Card Management

The MVP will provide full credit card management.

11.1 Credit Card Information

Each credit card must support:

- Credit Limit.
- Outstanding Balance.
- Available Credit.
- Statement Date.
- Payment Due Date.
- Minimum Payment.
- Payment Tracking.

11.2 Credit Card Transactions

Credit card purchases are recorded as Expenses.

Payments from cash or bank accounts to credit cards are recorded as Transfers.

A credit card payment must not create a second Expense.

11.3 Payments and Reminders

Users can manually record credit card payments.

The application will provide reminders before payment due dates.

11.4 Accounting Considerations

Credit cards represent liabilities, not ordinary positive-balance accounts.

The financial calculation engine must handle credit card balances separately from cash and bank balances.

Available Credit must be calculated consistently from the credit limit and outstanding balance.

---

12. Debt and Loan Management

The MVP will support:

- Personal Loans.
- Car Loans.
- Installment Plans.
- Purchase Installments.

12.1 Debt Information

Each debt record must include:

- Debt Name.
- Debt Type.
- Original Amount.
- Remaining Amount.
- Monthly Installment.
- Next Payment Date.
- Payment History.

12.2 Calculation Model

The MVP will use Simple Tracking.

Automatic interest calculations and detailed amortization schedules are deferred.

Users can manually update remaining debt amounts when necessary.

12.3 Debt Payment Accounting

The application must avoid double-counting principal repayment as an Expense.

Debt repayments must distinguish between liability reduction and actual expenses such as interest or fees when those are recorded.

---

13. Budget Management

13.1 Budget Period

The MVP supports monthly budgets.

Budgets are configured per category.

Weekly, annual, and custom-period budgets are deferred.

13.2 Budget Information

Each budget includes:

- Category.
- Monthly Budget Amount.
- Amount Spent.
- Remaining Amount.
- Usage Percentage.

13.3 Budget Alerts

The application will generate alerts at:

- 50%.
- 75%.
- 90%.
- 100%.

Each threshold must trigger only once per budget per month.

Refunds do not reduce budget consumption in the MVP.

---

14. Recurring Transactions

The application will support automatic recurring transaction generation.

14.1 Recurring Transaction Fields

- Amount.
- Account.
- Category.
- Frequency.
- Start Date.
- Optional End Date.
- Next Execution Date.
- Active/Paused Status.

14.2 Execution Requirements

Background jobs will generate transactions according to their schedules.

The system must support idempotent execution.

A background job retry or repeated execution must never generate duplicate transactions.

---

15. Bills and Subscriptions

The application will provide separate Bills and Subscriptions Management.

Examples:

- Electricity.
- Water.
- Internet.
- Mobile Bills.
- Netflix.
- Other Subscriptions.

15.1 Bill Information

Each bill includes:

- Name.
- Amount.
- Currency.

- Due Date.
- Frequency.
- Category.
- Payment Status.
- Reminder Settings.

Supported frequencies:

- Monthly.
- Quarterly.
- Yearly.

Supported payment statuses:

- Paid.
- Unpaid.
- Overdue.

15.2 Bill Payments

Users can:

- Create a transaction directly from a bill.
- Link an existing transaction to a bill.
- Mark a bill as Paid.
- View payment history.

The application must prevent duplicate expense recording.

15.3 Bills vs. Recurring Transactions

These features have different purposes:

- Recurring Transactions automatically create financial movements.
- Bills track financial obligations and due dates.

The system must preserve this distinction.

---

16. Financial Dashboard

The MVP will provide a comprehensive dashboard with a fixed layout.

Users cannot customize dashboard sections in the MVP.

16.1 Financial Overview

Display:

- Total Assets.
- Total Liabilities.
- Net Worth.
- Total Income.
- Total Expenses.

16.2 Cash Flow

Display:

- Current Account Balances.
- Monthly Income vs. Expenses.
- Upcoming Bills.
- Recent Transactions.

16.3 Budgets

Display:

- Budget Usage.
- Remaining Budget.
- Budget Alerts.

16.4 Credit Cards

Display:

- Outstanding Balances.
- Available Credit.
- Upcoming Due Dates.

16.5 Debts and Loans

Display:

- Total Remaining Debt.
- Monthly Installments.
- Next Payment Dates.

16.6 Financial Insights

Display:

- Spending Trends.
- Highest Spending Categories.
- Changes in Net Worth.

All aggregated values must use the user's Base Currency.

Individual account balances remain displayed in their original currencies.

---

17. Financial Reports

The MVP will support monthly financial reports.

17.1 Monthly Report Contents

- Total Income.
- Total Expenses.
- Net Cash Flow.
- Spending by Category.
- Budget Performance.
- Monthly Savings.
- Monthly Income and Expense Comparison.

Annual reports and custom date ranges are deferred.

Financial reports must use preserved historical exchange rates.

---

18. Notifications

The application will support:

- In-App Notifications.
- Email Notifications.

18.1 Notification Types

- Budget Alerts.
- Upcoming Bills.
- Credit Card Due Dates.

- Loan Installments.
- Savings Goal Updates.

18.2 Notification Preferences

Users must be able to control notification types.

Specific notification timing and reminder schedules will be defined during detailed feature design.

---

19. Global Search

The application will provide a global search experience.

Searchable entities:

- Transactions.
- Accounts.
- Categories.
- Bills.
- Debts.
- Savings Goals.

Search results must respect user-level data isolation.

---

20. Offline Support and PWA

20.1 PWA Requirements

The application must be installable as a Progressive Web App.

It must support:

- Web App Manifest.
- Service Worker.
- Installability.
- App Icon.
- Splash Screen.
- Standalone App Experience.
- Responsive Mobile-First UI.
- Touch-Friendly Interactions.

20.2 Offline Capabilities

Users must be able to:

- View saved accounts and balances.
- View previously downloaded transactions.
- Add Income transactions.
- Add Expense transactions.
- Create Transfers.
- Edit permitted records.

20.3 Local Database

IndexedDB will be used for local application data.

A synchronization queue will track offline changes.

20.4 Automatic Synchronization

When connectivity returns, the application will synchronize pending changes with the server.

The synchronization system must support:

- Idempotency Keys.
- Retry Handling.
- Conflict Resolution.
- Duplicate Prevention.
- Sync Status Tracking.

Unsynchronized transactions must be marked as `Pending Sync`.

Pending transactions must not be represented as server-confirmed financial records.

20.5 Local Data Encryption

Encrypted Local Storage is required.

The proposed approach includes:

- IndexedDB.
- Web Crypto API.
- Secure Key Management.
- HTTPS Communication.

Encryption keys must not be stored beside the encrypted data in a way that defeats the protection.

The final key derivation and storage design must be validated during architecture and security planning.

The application must clear sensitive local data on logout according to the agreed security policy.

Browser-based encryption does not provide the same security guarantees as native Secure Enclave or Android Keystore hardware-backed storage.

20.6 Conflict Resolution

The system must define how conflicting offline and server-side changes are resolved.

Financial records must never be silently overwritten in a way that loses or duplicates financial activity.

20.7 Automatic App Lock

The application must lock after five minutes of inactivity.

Users must re-authenticate to continue.

Pending synchronization data must not be lost when the application locks.

20.8 Biometric Authentication

Biometric authentication will be used for app unlocking where supported.

Supported device capabilities may include:

- Fingerprint.
- Face ID.

The application will use platform authenticator capabilities where available.

If biometric authentication is unavailable, users must be able to re-authenticate using their account credentials.

An application-specific PIN is not included in the MVP.

---

21. Appearance and User Experience

21.1 Themes

The application will support:

- Light Mode.
- Dark Mode.

Users manually select their preferred theme.

The preference must persist between sessions.

Automatic System Preference selection is not included in the MVP.

21.2 Language

The MVP will be English-only.

The application architecture must allow future Arabic language support and RTL layouts without major architectural changes.

---

22. Data Export and Privacy

22.1 Export Formats

Users can export their financial information in:

- CSV.
- Excel.
- PDF.

22.2 Export Scope

Exports must include the user's own data only.

Transaction exports must preserve:

- Original Amount.
- Original Currency.
- Base Currency Conversion.
- Exchange Rate Information.

22.3 Backups

There will be no user-facing backup or restore feature in the MVP.

However, operational database backups are mandatory.

Backup and recovery procedures must be established before Production launch.

---

23. AI CFO — Phase 2

The AI CFO is intentionally deferred until the core financial engine has been implemented and validated.

Potential future capabilities:

- Spending Analysis.
- Financial Recommendations.
- Budget Optimization.
- Cash Flow Predictions.
- Savings Recommendations.
- Natural-Language Financial Questions.

23.1 AI Safety Principle

The Financial Engine must remain the authoritative source of financial calculations.

AI may analyze and explain financial information.

AI must not autonomously move money, modify financial records, or execute financial actions.

---

24. Pricing and Business Model

The MVP will be completely free.

The MVP will not include:

- Paid Plans.
- Premium Features.
- Subscription Fees.
- Billing System.
- Payment Gateway.

Future monetization decisions are outside the current product scope.

25. Launch Strategy

The product will follow a Private Launch strategy.

Stage 1 — Internal Testing

Validate:

- Accounts and balances.
- Transactions and transfers.
- Currency conversion.
- Offline synchronization.
- Authentication and authorization.
- Security.
- Financial calculations.

Stage 2 — Private Beta

Target: **20–50 invited users**.

Objectives:

- Collect user feedback.
- Identify usability issues.
- Detect calculation errors.
- Monitor synchronization reliability.
- Identify performance problems.
- Validate the overall user experience.

Stage 3 — Public Launch

Open registration to the public after:

- Critical issues are resolved.
- Financial calculations are validated.
- Security checks are completed.
- Offline synchronization is sufficiently reliable.
- Production deployment and backup procedures are verified.

26. Development and Deployment Strategy

26.1 Hosting Approach

The product will use Managed Hosting Services.

This approach reduces infrastructure management overhead and allows the development team to focus on product implementation.

The exact service providers will be selected during architecture planning.

Proposed hosting responsibilities:

Component	Hosting Requirement
Frontend / PWA	Managed Frontend Hosting
Backend API	Managed Backend Hosting
Database	Managed PostgreSQL
File Storage	Private Object Storage
Background Jobs	Managed Worker
Email	Transactional Email Provider

26.2 Environments

The project will use three environments:

Development

Used for:

- Daily development.
- Feature implementation.
- Local testing.

Staging

Used for:

- Pre-production validation.
- Integration testing.
- Release verification.

Production

Used by real users.

Each environment must have separate databases, credentials, and secrets.

Real user data must not be copied into Development.

26.3 CI/CD

Automated CI/CD will be used.

The proposed workflow:

1. Developer pushes code.
2. Automated checks run.
3. Unit tests execute.
4. Build and validation checks execute.
5. Changes are deployed to the appropriate environment.
6. Staging is validated.
7. Production deployment requires manual approval.

Database migrations must be reviewed and tested before Production deployment.

Rollback procedures must be available.

---

27. Testing Strategy

The MVP will use Basic Testing, consisting of Unit Tests and Manual Testing.

However, critical financial operations require mandatory automated tests.

27.1 Unit Tests

Test important calculations and business rules, including:

- Balance Calculations.
- Currency Conversion.
- Budget Calculations.
- Transaction Impact.
- Net Worth Calculations.
- Debt and Credit Card Calculations.

27.2 Manual Testing

Manually validate:

- User Workflows.
- Responsive Layout.
- PWA Installation.
- Offline Mode.
- Synchronization.
- Authentication.
- Financial Reports.

27.3 Critical Financial Scenarios

The following scenarios must have automated regression coverage:

- Transfers must not count as Income or Expenses.
- Credit card payments must not create duplicate Expenses.
- Transaction editing must correctly reverse and reapply balance changes.
- Soft deletion must correctly reverse financial impact.
- Duplicate requests must not create duplicate financial records.
- User data must remain isolated.
- Offline synchronization must not duplicate transactions.
- Financial calculations must preserve currency precision.

---

28. Monitoring and Logging

The MVP will use Basic Monitoring.

The system must capture:

- Server Errors.
- Application Logs.
- Database Errors.
- Deployment Logs.

Logs must not expose:

- Passwords.
- Authentication Tokens.
- API Secrets.
- Sensitive Financial Information.

Advanced Performance Monitoring and sophisticated Alerting are deferred.

---

29. Documentation Requirements

Comprehensive technical documentation is required.

The project must maintain:

1. README.
2. Setup Instructions.
3. API Documentation.
4. Architecture Documentation.
5. Database Schema Documentation.
6. Deployment Guide.
7. Security Documentation.
8. Testing Guide.

Documentation must be updated as the implementation evolves.

---

30. Non-Functional Requirements

The application must be designed around the following quality attributes:

Security

Protect user authentication, financial information, files, and application access.

Data Integrity

Financial calculations and account balances must remain consistent.

Reliability

The application must handle failed requests, background job retries, and network interruptions safely.

Maintainability

The architecture must remain understandable and easy to extend.

Scalability

The application should support growth beyond the initial private beta without requiring an immediate architectural rewrite.

Usability

The experience must be clear, responsive, and accessible across desktop and mobile devices.

Offline Reliability

Offline changes must be tracked and synchronized without losing or duplicating financial records.

---

31. Critical Business Rules

The following rules are fundamental to the product:

1. Opening Balance is not Income.
2. Transfers are not Income or Expenses.
3. Credit card purchases are Expenses.
4. Credit card payments are Transfers.
5. Savings goal funding is a Transfer, not an Expense.
6. Refunds are Income under the Refunds category.
7. Refunds do not reduce budget consumption in the MVP.
8. Historical transactions use historical exchange rates.
9. Exchange rates used by transactions must be preserved.
10. All financial balance changes must be atomic.
11. Transaction deletion must use Soft Delete.
12. Financial audit history must be preserved.
13. Users can only access their own financial data.
14. Admin access does not automatically grant access to private financial records.
15. Offline transactions remain Pending Sync until confirmed by the server.
16. Recurring jobs and synchronization must be idempotent.
17. Financial calculations must be owned by the Financial Engine, not AI.

---

32. Open Technical Decisions

The following technical decisions have not yet been finalized.

They must be addressed during System Architecture planning.

Area	Candidate / Question
Frontend	Next.js + TypeScript
Backend	Next.js Backend or Separate Node.js/TypeScript API
Database	PostgreSQL

Area	Candidate / Question
ORM	Prisma or an alternative
Validation	Zod
Charts	Recharts or an alternative
Authentication	Authentication provider or custom implementation
File Storage	Managed Private Object Storage
Background Jobs	Managed Worker / Job Queue
Exchange Rates	Select a reliable provider
Email	Select a transactional email provider
Hosting	Select specific managed providers
CI/CD	GitHub Actions proposed
Monitoring	Basic logs and error tracking
Offline Sync	Define synchronization and conflict-resolution architecture

These are candidates, not approved technology decisions.

---

33. MVP Success Criteria

The MVP will be considered ready for private beta when:

- Users can securely register and authenticate.
- Each user can manage independent financial accounts.
- Transactions correctly update balances.
- Transfers do not distort income and expense calculations.
- Multi-currency calculations work correctly.
- Historical exchange rates are preserved.
- Credit cards and debts are represented correctly.
- Budgets and reports produce consistent results.
- Recurring transactions do not generate duplicates.
- Bills can be paid without duplicate expenses.
- Offline transactions synchronize reliably.
- Pending synchronization states are clearly communicated.
- User data isolation is verified.
- PWA installation works on supported devices.
- Critical financial regression tests pass.
- Backup and recovery procedures are verified.
- Staging and Production deployment processes are operational.

34. Proposed Development Phases

The following is a proposed implementation sequence, not a fixed delivery schedule.

Phase 1 — Foundation

- Repository Setup.
- Development Environment.
- Project Structure.
- Authentication.
- Database Foundation.
- User Isolation.
- Core Architecture.

Phase 2 — Financial Engine

- Accounts.
- Opening Balances.
- Transactions.
- Categories.
- Transfers.
- Balance Management.
- Currency Conversion.

Phase 3 — Financial Management

- Budgets.
- Savings Goals.
- Credit Cards.
- Debts and Loans.
- Bills and Subscriptions.
- Recurring Transactions.

Phase 4 — User Experience

- Dashboard.
- Monthly Reports.
- Global Search.
- Notifications.
- Light and Dark Themes.
- Responsive Design.

Phase 5 — PWA and Offline

- PWA Installation.
- Local Database.
- Local Encryption.
- Sync Queue.
- Automatic Synchronization.
- Conflict Resolution.
- App Lock.
- Biometric Unlock Where Supported.

Phase 6 — Production Readiness

- Testing.
- Security Review.
- Monitoring.
- CI/CD.
- Documentation.
- Backups.
- Staging Validation.

Phase 7 — Private Beta

- Invite 20–50 Users.
- Collect Feedback.
- Fix Critical Issues.
- Validate Reliability.

Phase 8 — Public Launch

- Final Production Validation.
- Open Registration.
- Monitor Application Health.

---

35. Final Product Definition

The Personal Finance App MVP is a free, secure, multi-user Personal Finance Management PWA.

It provides comprehensive personal financial management, including accounts, transactions, transfers, multi-currency support, budgets, savings, credit cards, debts, loans, bills, subscriptions, reports, and offline synchronization.

The product prioritizes:

- Financial Accuracy.
- Data Integrity.
- Security.
- Reliability.
- Maintainability.
- User Experience.

AI-powered financial assistance will be introduced only after the core financial engine has been validated.

This document defines the approved MVP product scope. Technical implementation choices remain subject to the System Architecture phase.